

Semantic Governance

(Vector DB - Context Engine)

The following is a design for building vector databases to capture domain-specific context for guiding a sales operations agent—one constrained by a domain-specific ontology and tasked with mapping ontology classes to schema ontologies for digital assets—requires an architecture that integrates several nuanced components and processes.

Core Architecture Overview

The database should be a hybrid system, combining:

- Vector DB (Domain) A vector store for semantic retrieval
- Mapping Ontology (Schema) - Relational/graph storage for explicit mappings and relations
- Metadata and embedding support for context-rich queries

Key Components and Data Flows

- **Ontology Entities Table:** Stores classes and properties from both the sales operations domain ontology and schema ontologies for digital assets. Includes fields for entity type, metadata, embedding (vector representation), and links to sub-domains.
- **Embeddings Index:** Each ontology class/property (e.g., “Lead,” “Account,” “Opportunity,” “Contract” in sales ops) is embedded into a vector space, using semantic embedding models suitable for domain-specific alignment. **This supports the approximate nearest neighbor (ANN) search for mapping discovery.**
- **Mapping Table:** Explicit links between domain ontology classes and schema ontology classes representing digital assets, with fields for:
 -  Source class
 -  Target class
 -  Mapping confidence (numeric or categorical)
 -  Mapping type (equivalent, broader, narrower) (relation from Upper Ontology)
 -  Validation status and history.
- **Sub-domain Context Index:** Sales operations consists of sub-domains (e.g., lead management, quoting, order fulfillment, pricing, sales analytics). Each sub-domain has distinct classes that may overlap or intersect. The index tracks associations, context vectors and provides scope filtering for querying and agent guidance.

Semantic Governance

(Vector DB - Context Engine)

Flow of Activities

1. **Entity Extraction:** Upon agent request, sales ops ontology classes are extracted and embedded. Metadata (name, description, relationships, sub-domain context) is stored.
2. **Semantic Mapping:** Matches between domain ontology classes and digital asset schema classes are proposed using vector similarity (semantic matching), then refined/ranked with syntactic, lexical, and structural features.
3. **Mapping Validation:** High-confidence mappings are validated and persisted with evidence; ambiguous mappings are flagged for review. Mappings are linked with the sub-domain context for traceability.
4. **Sub-domain Navigation:** Agent queries can be scoped by sub-domain (e.g., “Order Fulfillment”), narrowing mapping search to relevant ontology sections.

Vector DB Schema Outline

Table	Key Fields	Purpose
Ontology_Entities	entity_id, name, description, type, embedding, subdomain	Stores ontology classes/properties with vectors and context
Schema_Entities	schema_id, name, type, description, embedding	Digital asset schema classes/properties
Entity_Mappings	mapping_id, source_entity_id, target_entity_id, confidence, type, status, evidence	Stores class-to-class mappings with context and validation
Subdomain_Index	subdomain_id, name, entities	Allows filtering and contextual search for agent queries

Domain Relevancy (5 Ws)

There are 5 Dimensions of Relevancy used to support Contextual Relevance:

- **Who** - Persona Model
- **What** - Business Domains & Sub-Domains
- **When** - Time Model
- **Where** - Location Model
- **Why** - Intent/Causality Model (see breakout below)

Semantic Governance

(Vector DB - Context Engine)

Intent Patterns - Overview

Intent patterns represent the underlying purpose or reasoning behind a user's query, such as:

- Diagnostic: What happened? Root cause analysis, problem identification.
- Evaluative: Assessments, service reviews, performance measurements.
- Predictive: Future outcomes, forecasts, trend anticipations.
- Reasoning Chains: Forward (cause-effect) or backward (goal-driven) reasoning.

Intent classification involves detecting these patterns using AI models that analyze query semantics, context, sentiment, and subtleties rather than relying on strict keywords.

Model-Specific Elements to Capture in Vector DB for WHY Dimension

Element	Description	Data Type / Representation	Purpose in Vector DB
Intent Embedding	Dense vector embedding of the detected intent pattern using fine-tuned NLP models	Float32 vector (e.g., 768-dim)	Enable semantic similarity search for intent clusters
Intent Category	Discrete categorical tag (diagnostic, evaluative, predictive, chaining, etc.)	Enum/String	Quick filtering and facet-style query refinement
Confidence Score	Model probability or certainty for the assigned intent	Float	Filter uncertain classifications, rank by confidence
Contextual Features	Embeddings or metadata capturing dialogue context influencing intent (preceding turns	Float32 vector or JSON metadata	Incorporates multi-turn conversation

Semantic Governance

(Vector DB - Context Engine)

	etc.)		understanding
Temporal Intent	Indicates if intent refers to past events, current assessment, or future predictions	Enum (Past, Present, Future)	Supports temporal filtering and reasoning
Intent Relationships	Links or references to related intents (e.g., sub-intents, related tasks)	URIs or IDs linked to intent ontology	Enables reasoning and chaining queries
Provenance Metadata	Source, extraction method, timestamp of intent detection	JSON metadata	Traceability and auditability

Patterns for SPARQL Generation & Intent Evaluation

- Use Intent Ontologies defining classes like `:DiagnosticIntent`, `:EvaluativeIntent`, `:PredictiveIntent`, and properties linking to domain concepts, temporal frames, and personas.
- SPARQL Templates parameterized by intent class guide query construction. For example:
 - Diagnostic intent queries include causal chains, event timelines.
 - Predictive intents query temporal projections or forecast models.
 - Evaluative intents use aggregation and scoring predicates.
- Combine vector similarity scoring on intent embeddings with SPARQL filter results for multi-modal ranking.
- Leverage property paths and reasoning rules for forward/backward chaining queries reflecting intent-driven deduction/inference.

Implementing Intent Pattern Capture in a Vector DB Pipeline

1. Intent Detection: NLP models (transformers, LSTMs) classify intents and generate dense embeddings from user input.
2. Dimensional Tagging: Assign categorical intent labels, temporal indicators, and confidence scores.

Semantic Governance

(Vector DB - Context Engine)

3. Indexing: Store embeddings and metadata in the vector DB with inverted indices on categories and temporal tags.
4. Search and Ranking: Query by embedding similarity refined by intent tags; combine with contextual filters (Who, What, When, Where).
5. SPARQL Query Generation: Use detected intent patterns to select and instantiate SPARQL templates on linked knowledge graphs.
6. Composite Relevance Scoring: Fuse vector-based semantic relevance with logic-based SPARQL results.

Summary

Capturing the WHY dimension's intent patterns in a vector DB requires both dense semantic embeddings and categorical, contextual metadata to represent reasoning nuances. This enables semantic search for intent similarity, precise categorizations, and layered relevance scoring. Tightly integrating this with linked ontologies and SPARQL query templates allows expressive, intent-aware retrieval that directly supports the purpose behind user queries, essential for conversational AI contextualization

Semantic Governance

(Vector DB - Context Engine)

Improve Search and Relevancy

- Encode ontological data as dense embeddings, mapping classes, relations, and instances into vector representations using IEEE-endorsed neural models (such as semantic encoders or graph neural networks), ensuring compatibility with OWL2/JSON-LD.
- Use the OMG Data Products Ontology (DPROD) spec to describe data products, metadata, entitlements, and data lineage for search and discovery, leveraging controls for compliance and distributed access.
- Apply RDF and DCAT vocabularies to structure entity metadata, ground semantic search, and facilitate cross-platform discovery and interoperability.
- Enable hybrid search and reranking pipelines that combine symbolic reasoning (OWL2/DOL-based inference) and vector similarity (cosine distance, reranked by ontological rules), guided by scenario-specific ontologies.
- Implement continuous integration and quality assurance for ontological objects following ISO standards for data quality, error reduction, and extensibility.

Implementation Outline

Component	Specification	Standard Ref
Ontology Design	ODM/OWL/RDF/JSON-LD for domain model and semantic mappings	OMG ODM, ISO/IEC/IEEE 24765, W3C RDF/OWL
Data Product Metadata	DCAT extension, DPROD schema for data products and lineage	OMG DPROD, W3C DCAT, ISO naming/versioning
Embedding Procedure	IEEE-endorsed deep graph encoders for semantic vectorization	IEEE ML/AI Guidance, OWL-API, ODM
Vector DB Integration	Schema alignment, rich metadata indexing, linkage to ontological URIs	OMG ODM, ISO/IEC JTC 1/SC 7, IEEE interoperability
Hybrid Retrieval System	Pipeline for vector search reranked by ontological inference	OMG DOL, ISO error reduction standards
Quality & Compliance	Automated tests, versioning, and data quality monitoring	ISO documentation, OBO Foundry practices
Scenario Mapping	Participatory ontology engineering, use-case embedding, modular workflows	OMG PSIG, domain-specific ontologies, W3C standards